

Deep Dive: Registry, SQLite & Event Logs

The upstream case, solved together — the two new challenges stay yours.

Stefanus Eko Prasetyo

Lab-2 · companion to the Lab 02 lecture

What we cover today

1 Registry in depth

hive anatomy, forensic hot spots, UserAssist ROT13, .reg exports

2 SQLite crash course

SELECT, WHERE, ORDER BY, COUNT - querying browser history like a pro

3 The Windows tool pack

reg.exe, wevtutil, Get-WinEvent, firefox_decrypt, LECmd

4 Worked examples

the upstream case: NTUSER.DAT, Firefox, and the PowerShell log

5 Your turn

Challenge B (the exported hive) and Challenge C (the history hunt)

Anatomy of a hive

Concept	Meaning
Hive file	the on-disk database: NTUSER.DAT, UsrClass.DAT, SAM, SYSTEM, SOFTWARE, SECURITY
Key / subkey	a node in the tree - like a folder path
Value = data	a named entry inside a key - like a file; types REG_SZ, REG_DWORD, REG_BINARY, REG_QWORD
Transaction logs	NTUSER.DAT.LOG1 / .LOG2 - undo data; can recover recent changes
Path grammar	root\subkey\subkey\ValueName

Two hives per user

- ▶ NTUSER.DAT - the main hive: Software, Environment, most Explorer settings
- ▶ UsrClass.DAT (under AppData\Local\Packages) - per-package state, shell bags
- ▶ a hive that will not mount may still yield keys via its .LOG1/.LOG2 or the Registry Explorer transaction view

Forensic hot spots under HKCU

Path (abbreviated)	Records	Watch out for
...\CurrentVersion\Run	logon autostart (user scope)	names that mimic Microsoft products
HKLM ...\Run + RunOnce	machine-wide autostart	compare both scopes
...\Explorer\StartupApproved\Run	which Run entries are enabled	disabled entry = recent change
...\Explorer\TypedPaths	paths typed in Explorer (MRU)	url1 is the MOST RECENT
...\Explorer\UserAssist	GUI executions + counts	value names are ROT13
...\Explorer\MountPoints2	mounted volumes incl. USB	serial number survives wiping
...\Explorer\RecentDocs	recently used documents (MRU)	binary values; use a parser

Querying hives from the console

PS C:\Tools>

```
PS> reg query "HKCU\Software\Microsoft\Windows
      \CurrentVersion\Run"
      Malware      REG_SZ      C:\Users\w\Desktop\malware.exe

PS> reg query HKCU\...\TypedPaths /v url1
      url1      REG_SZ      C:\Windows\System32\calc.exe

PS> reg export HKCU\...\Run run_backup.reg
# .reg exports are plain text - grep-able with Lab 01 skills
PS> Select-String -Path suspect.reg -Pattern "Run"
```

reg.exe essentials

- ▶ reg query <key> - list subkeys and values
- ▶ reg query <key> /v Name - one value
- ▶ reg query <key> /s - recurse everything below
- ▶ reg export / reg import - snapshots and text form
- ▶ for OFFLINE hives use Registry Explorer instead - reg.exe reads the LIVE registry

UserAssist — executions, ROT13-flavored

PS C:\Tools>

```
PS> reg query HKCU\...\UserAssist\{CEBFF5CD...}\Count /s
      HRZR_PUERNQ ...
      HRZR_HACFVQR

# ROT13 decode (powershell one-liner):
PS> 'HRZR_HACFVQR' | % {
    -join ($_ | % {[char](([int]$_ - 65 + 13) % 26 + 65)})
}
UEME_BACKGROUNDM

# decoded UEME_RUNPATH entries = executed programs + run counts
```

Reading UserAssist

- ▶ value names hide program paths in ROT13 - obfuscation, NOT encryption
- ▶ UEME_RUNPATHC:... = program path; run count and focus time in the binary data
- ▶ GUI-launched programs only (not cmd.exe scripts)
- ▶ counts prove USE, not mere presence on disk

Recognizing persistence on sight

Pattern	Looks like	Why suspicious
Masquerading name	WindowsDefenderUpdate, SecurityHealth	mimics Microsoft naming
Interpreter + encoded	powershell -enc <base64>	hidden payload, no readable command
Odd user-writable path	C:\Users\w\Desktop\malware.exe	autostart should live in Program Files
Enabled-but-fresh	StartupApproved entry recently toggled	new autostart = new persistence

Habit to build

- ▶ on every case: enumerate Run + RunOnce (HKCU and HKLM), then StartupApproved for each entry
- ▶ ask three questions: is the name real? is the path sane? is the command readable?

SELECT and WHERE — asking the database

```
PS> sqlite3 places.sqlite
sqlite> .tables
moz_places  moz_historyvisits
sqlite> SELECT url, visit_count FROM moz_places
...> WHERE url LIKE '%github%';
https://www.github.com/|16
sqlite> SELECT url FROM moz_places
...> WHERE url LIKE '%paste%';
https://paste.example/raw/ab12cd34
```

The pattern

- ▶ SELECT columns FROM table
- ▶ WHERE filters rows (LIKE '%x%' = contains x)
- ▶ strings use single quotes; statements end with ;
- ▶ DB Browser gives the same power with clicks

ORDER BY, COUNT, and the strange timestamps

PS> sqlite3

```
sqlite> SELECT url, visit_count FROM moz_places
...> ORDER BY visit_count DESC LIMIT 5;
sqlite> SELECT place_id, visit_date FROM moz_historyvisits
...> ORDER BY visit_date DESC LIMIT 3;

# PRTime = microseconds since 1601-01-01 (Firefox epoch)
# unix = PRtime / 1_000_000 - 11644473600
sqlite> SELECT datetime(visit_date/1000000,
...> 'unixepoch') FROM moz_historyvisits LIMIT 1;
```

Aggregates

- ▶ COUNT(*), GROUP BY - frequency tables
- ▶ ORDER BY x DESC LIMIT n - top-N lists
- ▶ JOIN matches visits to their places by id
- ▶ always convert PRTime before reading times

Rebuilding an evening from history

```
sqlite> SELECT datetime(v.visit_date/1000000, 'unixepoch') AS t,  
...>      p.url  
...> FROM moz_historyvisits v JOIN moz_places p ON p.id = v.place_id  
...> ORDER BY v.visit_date DESC LIMIT 8;
```

```
2026-09-20 22:47:09 | forum.example.net/thread/8812?page=4  
2026-09-20 22:38:02 | files.example/drop/setup_v2.exe  
2026-09-20 22:33:41 | google.com/search?q=how+to+remove+...  
2026-09-20 22:26:14 | paste.example/raw/ab12cd34  
# searches -> forum -> paste -> download: intent becomes timeline
```

PS> sqlite3

Event log querying from the console

PS C:\Tools>

last 20 events of a channel

```
PS> Get-WinEvent -LogName 'Windows PowerShell/Operational'  
-MaxEvents 20
```

only Event ID 4104, newest first

```
PS> Get-WinEvent -FilterHashtable @{  
    LogName='Windows PowerShell/Operational'; Id=4104 }
```

classic exe:

```
PS> wevtutil qe Security /q:"*[System[(EventID=4625)]]" /c:10 /rd:true
```

Choosing the tool

- ▶ Event Viewer - exploring, one log at a time
- ▶ Get-WinEvent - modern, filterable, pipelineable
- ▶ wevtutil - the classic, terse, everywhere
- ▶ FilterHashtable is the fast one (server-side filter)

LECmd and firefox_decrypt



PS C:\Tools>

```
PS> .\LECmd.exe -f .\passwd.lnk --csv . --csvf lnk.csv  
# dumps target path, timestamps, machine ID, MAC to CSV
```

```
PS> git clone https://github.com/unode/firefox_decrypt  
PS> python firefox_decrypt.py .\s6upyltd.default-release  
Website: https://www.reddit.com  
Username: 'hackerman' Password: 'sup3rs3cur3p4ssw0rd'  
# needs key4.db + logins.json from the SAME profile
```

NTUSER.DAT — three answers

```
Control Panel\Mouse -> DoubleClickSpeed = 500

...\Explorer\TypedPaths -> url1 = C:\Windows\System32\calc.exe
# url1 = the MOST RECENT path typed into Explorer

...\CurrentVersion\Run ->
    OneDrive = "...OneDrive.exe" /background      (legit)
    Malware   = C:\Users\w\Desktop\malware.exe     (persistence!)

# a value named Malware is a gift; real intruders pick nicer names
```

PS C:\Tools>

Firefox — history and saved logins

```
sqlite> -- top domain by URL count:
sqlite> SELECT host, COUNT(*) c FROM (
...>   SELECT substr(url, instr(url,'//')+2),
...>         substr(...,1,instr(...||'/', '/')-1) AS host FROM moz_places)
...> GROUP BY host ORDER BY c DESC LIMIT 3;
www.amazon.com | 10      <- most visited site
www.mozilla.org | 5

PS> python firefox_decrypt.py <profile>
Website: https://www.reddit.com
Username: 'hackerman'    Password: 'sup3rs3cur3p4ssw0rd'
```

PS C:\Tools>

The PowerShell log — full story, one flag

```
PS C:\Tools>
# 4104 blocks in order:
New-Item ...\ScriptBlockLogging -Force
Set-ItemProperty EnableScriptBlockLogging = 1
Invoke-WebRequest ... file.ps1 -OutFile file.ps1
del .\file.ps1 (then again, twice)
.\file.ps1 <- execution

# file.ps1 decodes base64 ->
"use flag{evcnt_logs_f0r_th3_w1n} as the answer"
```

Upstream flag (original format):

```
flag{evcnt_logs_f0r_th3_w1n}
```

Narrative

- logging enabled, payload pulled, cleaned up, re-pulled, executed

Two fresh artifacts. Same discipline.

01

Challenge B — 4.2

suspect_ntuser.reg: one Run value hides in plain sight among legitimate ones. Decode the encoded command, un-ROT13 the UserAssist trail, and identify the mounted USB device.

02

Challenge C — 4.3

places.sqlite: 310 rows of history with one suspicious evening. Rank the domains, rebuild the timeline, and decode the search that should never have been made.

03

Working paper

pre-lab first; staged hints in section 4; evidence numbering B-xx; hashes before and after.